

Source Code Summarization in the Era of Large Language Models

Weisong Sun¹, Yun Miao², Yuekang Li³, Hongyu Zhang⁴, Chunrong Fang², Yi Liu¹, Gelei Deng¹,
Yang Liu¹, Zhenyu Chen²

¹College of Computing and Data Science, Nanyang Technological University Singapore, Singapore

²State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China

³School of Computer Science and Engineering, University of New South Wales, Sidney, Australia

⁴School of Big Data and Software Engineering, Chongqing University, Chongqing, China

weisong.sun@ntu.edu.sg, miaoyun001my@gmail.com, yuekang.li@unsw.edu.au, hyzhang@cqu.edu.cn,
fangchunrong@nju.edu.cn, yi009@e.ntu.edu.sg, gelei.deng@ntu.edu.sg, yangliu@ntu.edu.sg, zychen@nju.edu.cn

Abstract—To support software developers in understanding and maintaining programs, various automatic (source) code summarization techniques have been proposed to generate a concise natural language summary (i.e., comment) for a given code snippet. Recently, the emergence of large language models (LLMs) has led to a great boost in the performance of code-related tasks. In this paper, we undertake a systematic and comprehensive study on code summarization in the era of LLMs, which covers multiple aspects involved in the workflow of LLM-based code summarization. Specifically, we begin by examining prevalent automated evaluation methods for assessing the quality of summaries generated by LLMs and find that the results of the GPT-4 evaluation method are most closely aligned with human evaluation. Then, we explore the effectiveness of five prompting techniques (zero-shot, few-shot, chain-of-thought, critique, and expert) in adapting LLMs to code summarization tasks. Contrary to expectations, advanced prompting techniques may not outperform simple zero-shot prompting. Next, we investigate the impact of LLMs’ model settings (including top_p and temperature parameters) on the quality of generated summaries. We find the impact of the two parameters on summary quality varies by the base LLM and programming language, but their impacts are similar. Moreover, we canvass LLMs’ abilities to summarize code snippets in distinct types of programming languages. The results reveal that LLMs perform suboptimally when summarizing code written in logic programming languages compared to other language types (e.g., procedural and object-oriented programming languages). Finally, we unexpectedly find that CodeLlama-Instruct with 7B parameters can outperform advanced GPT-4 in generating summaries describing code design rationale and asserting code properties. We hope that our findings can provide a comprehensive understanding of code summarization in the era of LLMs.

Index Terms—large language model, source code summarization, prompt engineering

I. INTRODUCTION

Code comments are vital for enhancing program comprehension [1] and facilitating software maintenance [2]. While it is considered good programming practice to write high-quality comments, the process is often labor-intensive and time-consuming [2]–[4]. As a result, high-quality comments are frequently absent, mismatched, or outdated during software evolution, posing a common problem in the software industry [5]–[7]. Automatic code summarization (or simply,

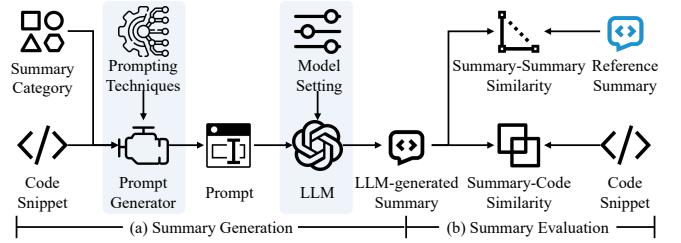


Fig. 1: General workflow of LLM-based code summarization and its effectiveness evaluation

code summarization), a hot research topic [8]–[10], addresses this challenge by developing advanced techniques/models for automatically generating natural language summaries (i.e., comments) for code snippets, such as Java methods or Python functions, provided by developers.

Recently, with the success of large language models (LLMs) in natural language processing (NLP) [11], an increasing number of software engineering (SE) researchers have started integrating them into the resolution process of various SE tasks [12], [13]. In this study, we focus on the application of LLMs on the code summarization tasks. Figure 1 shows the general workflow of LLM-based code summarization and its effectiveness evaluation methods. In the summary generation process, the input consists of a code snippet and the expected summary category. The input is passed to a prompt generator equipped with various prompt engineering techniques (referred to as prompting technique), which constructs a prompt based on input. This prompt is then used to instruct LLMs to generate a summary of the expected type for the input code snippet. In the summary evaluation process, a common method used to automatically assess the quality of LLM-generated summaries is to compute the text or semantic similarity between the LLM-generated summaries and the reference summaries.

There have been several recent studies investigating the effectiveness of LLMs in code summarization tasks [14]–[18]. These studies can help subsequent researchers rapidly understand the aspects of code summarization garnering attention in the era of LLMs, but they still have some limitations. First, most of them only focus on one prompting technique, while

some advanced prompting techniques have not been investigated and compared (e.g., chain-of-thought prompting). For example, Sun et al. [16] solely focus on zero-shot prompting, while several other studies [14], [17], [19] only focus on few-shot prompting. Second, they overlook the impact of the model settings (i.e., parameter configuration) of LLMs on their code summarization capabilities. There is no empirical evidence showing LLMs remain well in all model settings. Last but not least, these studies follow prior code summarization studies [4], [20], [21] to evaluate the quality of summaries generated by LLMs through computing text similarity (e.g., BLEU [22], METEOR [23], and ROUGE-L [24]) or semantic similarity (e.g., SentenceBERT-based cosine similarity [25]) between the LLM-generated summaries and the reference summaries, detailed in Section IV-A. However, prior research by Sun et al. [16] has shown that compared to traditional code summarization models, the summaries generated by LLMs significantly differ from reference summaries in expression and tend to describe more details. Consequently, whether these traditional evaluation methods are suitable for assessing the quality of LLM-generated summaries remains unknown.

To address these issues, in this paper, we conduct a systematic study on code summarization in the era of LLMs, which covers various aspects involved in the LLM-based code summarization workflow. Considering that the choice of evaluation methods directly impacts the accuracy and reliability of the evaluation results, we first systematically investigate the suitability of existing automated evaluation methods for assessing the quality of summaries generated by LLMs (including CodeLlama-Instruct, StarChat- β , GPT-3.5, and GPT-4). Specifically, we compare multiple automated evaluation methods (including methods based on summary-summary text similarity, summary-summary semantic similarity, and summary-code semantic similarity) with human evaluation to reveal their correlation. Inspired by the work in NLP [26]–[28], we also explore the possibility of using the LLMs themselves as evaluation methods. The experimental results show that **among all automated evaluation methods, the GPT-4-based evaluation method overall has the strongest correlation with human evaluation**. Second, we conduct comprehensive experiments on three widely used programming languages (Java, Python, and C) datasets to explore the effectiveness of five prompting techniques (including zero-shot, few-shot, chain-of-thought, critique, and expert) in adapting LLMs to code summarization tasks. The experimental results show that the optimal choice of prompting techniques varies for different LLMs and programming languages. **Surprisingly, the more advanced prompting techniques expected to perform better may not necessarily outperform simple zero-shot prompting**. For instance, when the base LLM is GPT-3.5, zero-shot prompting outperforms the other four more advanced prompting techniques overall on three datasets. Then, we investigate the impact of two key model settings/parameters, including top_p and temperature, on LLMs’ code summarization performance. These two parameters may affect the randomness of generated summaries. The results demonstrate

that **the effect of top_p and temperature on summary quality varies depending on the base LLM and programming language**. As alternative parameters, they exhibit a similar impact on the quality of LLM-generated summaries. Furthermore, unlike existing studies that simply experimented with multiple programming languages, we reveal the differences in the code summarization capabilities of LLMs across five types (including procedural, object-oriented, scripting, functional, and logic programming languages) encompassing ten programming languages: Java, Python, C, Ruby, PHP, JavaScript, Go, Erlang, Haskell, and Prolog. The Erlang, Haskell, and Prolog datasets are built by ourselves and we make them public to the community. We find that **across all five types of programming languages, LLMs consistently perform the worst in summarizing code written in logic programming languages**. Finally, we investigate the ability of LLMs to generate summaries of different categories, including What, Why, How-to-use-it, How-it-is-done, Property, and Others. The results reveal that the four LLMs perform well in generating distinct categories of summaries. For example, **CodeLlama-Instruct excels in generating Why and Property summaries, while GPT-4 is good at generating What, How-it-is-done, and How-to-use summaries**. Our comprehensive research findings will assist subsequent researchers in quickly and deeply understanding the various aspects involved in the workflow of code summarization based on LLMs, as well as in designing advanced LLM-based code summarization techniques for specific fields.

In summary, we make the following contributions.

- To the best of our knowledge, we conduct the first investigation into the feasibility of applying LLMs as evaluators to assess the quality of LLM-generated summaries.
- We conduct a thorough study of code summarization in the era of LLMs, covering multiple aspects of the LLM-based code summarization workflow, and come up with several novel and unexpected findings and insights. These findings and insights can benefit future research and practical usage of LLM-based code summarization.
- We make our dataset and source code publicly accessible [29] to facilitate the replication of our study and its application in extensive contexts.

II. BACKGROUND AND RELATED WORK

Code summarization is the task of automatically generating natural language summaries (also called comments) for code snippets. Such summaries serve various purposes, including but not limited to explaining the functionality of code snippets [7], [17], [30]. The research on code summarization can be traced back to as early as 2010 when Sonia Haiduc et al. [31] introduced automated text summarization technology to summarize source code. Later on, following the significant success of neural machine translation (NMT) research in the field of NLP [32], [33], a large number of researchers migrate its underlying encoder-decoder architecture to code summarization tasks [4], [8], [34]–[36]. In the past two years, research on LLM-based code summarization has mushroomed.

Fried et al. [37] introduce an LLM called InCoder, and try zero-shot training on the CodeXGLUE [38] Python dataset. InCoder achieves impressive results, but fine-tuned small PLMs like CodeT5 can still outperform the zero-shot setting. Ahmed et al. [14] investigate the effectiveness of few-shot prompting in adapting LLMs to code summarization and find that it can make Codex significantly outperform fine-tuned small PLMs (e.g., CodeT5). Given the concern of potential code asset leakage when using commercial LLMs (e.g., GPT-3.5), Su et al. [39] utilize knowledge distillation technology to distill small models from LLMs (e.g., GPT-3.5). Their experimental findings reveal that the distilled small models can achieve comparable code summarization performance to LLMs. Gao et al. [19] investigate the optimal settings for few-shot learning, including few-shot example selection methods, few-shot example order, and the number of few-shot examples. Geng et al. [17] investigate LLMs’ ability to address multi-intent comment generation. Ahmed et al. [40] propose to enhance few-shot samples with semantic facts automatically extracted from the source code. Sun et al. [16] design several heuristic questions to collect the feedback of ChatGPT, thereby finding an appropriate prompt to guide ChatGPT to generate in-distribution code summaries. Rukmono et al. [41] address the unreliability of LLMs in performing reasoning by applying a chain-of-thought prompting strategy. Recently, some studies [10], [15], [42] have also investigated the applicability of Parameter-Efficient Fine-Tuning (PEFT) techniques in code summarization tasks. In this paper, we focus on uncovering the effectiveness of various prompting techniques in adapting LLMs to code summarization without fine-tuning.

III. STUDY DESIGN

A. Research Questions

This study aims to answer the following research questions:

RQ1: What evaluation methods are suitable for assessing the quality of summaries generated by LLMs? Existing research on LLM-based code summarization [14], [17], [19] widely follow earlier studies [21], [25] and employ automated evaluation metrics (e.g., BLEU) to evaluate the quality of LLM-generated summaries. However, recent studies [16], [39] have shown that LLM-generated summaries surpass reference summaries in quality. Therefore, evaluating LLM-generated summaries based on their text or semantic similarity to reference summaries may not be appropriate. This RQ aims to discover a suitable method for automated assessment of the quality of LLM-generated summaries.

RQ2: How effective are different prompting techniques in adapting LLMs to the code summarization task? This RQ aims to unveil the effectiveness of several popular prompting techniques (e.g., few-shot and chain-of-thought) in adapting LLMs to code summarization tasks.

RQ3: How do different model settings affect LLMs’ code summarization performance? To better meet diverse user needs, LLMs typically offer configurable parameters (i.e., model settings) that allow users to control the randomness of model behaviour. In this RQ, we adjust the randomness of the

generated summaries by modifying LLMs’ parameters and see the impact of different model settings on the performance of LLMs in generating code summaries.

RQ4: How do LLMs perform in summarizing code snippets written in different types of programming languages? Programming languages are diverse in types (e.g., object-oriented and functional programming languages), with their implementations of the same functional requirements being similar or entirely different. The scale of programs implemented with them in Internet/open-source repositories also varies, which may result in differences in the mastery of knowledge of these languages by LLMs. Hence, this RQ aims to reveal the differences in LLMs’ capabilities to summarize code snippets across diverse programming language types.

RQ5: How do LLMs perform on different categories of summaries? Previous research [3], [43], [44] has shown that summaries can be classified into various categories according to developers’ intentions, including What, Why, How-to-use-it, How-it-is-done, Property, and others. Therefore, in this RQ, we aim to explore the ability of LLMs to generate summaries of different categories.

B. Experimental LLMs

We select four LLMs as experimental representatives.

CodeLlama-Instruct. Code Llama [45] is a family of LLMs for code based on Llama 2 [46]. It provides multiple flavors to cover a wide range of applications: foundation models, Python specializations (Code Llama-Python), and instruction-following models (Code Llama-Instruct) with 7B, 13B, 34B, and 70B parameters. In this study, we evaluate the summaries generated by CodeLlama-Instruct 7B. We also verify the ability of CodeLlama-Instruct 70B to act as an evaluator in RQ1.

StarChat- β . StarChat- β [47] is an LLM with 16B parameters fine-tuned on StarCoderPlus [48]. Compared with StarCoderPlus, StarChat- β excels in chat-based coding assistance.

GPT-3.5. GPT-3.5 [49] is an LLM provided by OpenAI. It is trained with massive texts and codes. It can understand and generate natural language or code.

GPT-4. GPT-4 is an improved version of GPT-3.5, which can solve difficult problems with greater accuracy. OpenAI has not disclosed the specific parameter scale of GPT-3.5 and GPT-4. Our study uses gpt-3.5-turbo and gpt-4-1106-preview.

Model Settings. Apart from RQ3 where we investigate the impact of model settings, we uniformly set the temperature to 0.1 to minimize the randomness of LLM’s responses and highlight the impact of evaluation methods/prompting techniques/programming language types/summary categories. Note that the LLM-generated summaries are relatively long, and usually consist of several sentences. By observing some examples, we find that the first sentences are suitable as the final summaries, and the following sentences elaborate on some details and supplementary explanations. Therefore, we extract the first sentences as the final summaries.

C. Prompting Techniques

We compare five commonly used prompting techniques below.

Zero-Shot. Zero-shot prompting adapts LLMs to downstream tasks using simple instructions. In our scenario, the input to LLMs consists of a simple instruction and a code snippet to be summarized. We expect LLMs to output a natural language summary of the code snippet. Therefore, we follow [16] and adopt the input format: *Please generate a short comment in one sentence for the following function: $\langle code \rangle$.*

Few-Shot. Few-shot prompting (also known as in-context learning [17], [19]) provides not only straightforward instruction but also some examples when adapting LLMs to downstream tasks. The examples serve as conditioning for subsequent examples where we would like LLMs to generate a response. In our scenario, the examples are pairs of $\langle code\ snippet, summary \rangle$. According to the findings of Gao et al. [19], we set the number of examples to 4 to achieve a balance between LLMs’ performance and the cost of calling the OpenAI API.

Chain-of-Thought. Chain-of-thought prompting adapts LLMs to downstream tasks by providing intermediate reasoning steps [50]. These steps enable LLMs to possess complex reasoning capabilities. In this study, we follow Wang et al. [51] and apply chain-of-thought prompting to the code summarization task through the following four steps:

- (1) Instruction 1: Input the code snippet and five questions about the code in the format
“Code: $\backslash n \langle code \rangle$
Question: $\backslash n \langle Q1 \rangle \backslash n \langle Q2 \rangle \backslash n \langle Q3 \rangle \backslash n \langle Q4 \rangle \backslash n \langle Q5 \rangle \backslash n$ ”
- (2) Get LLMs’ response to Instruction 1, i.e., Response 1.
- (3) Instruction 2: “Let’s integrate the above information and generate a short comment in one sentence for the function.”
- (4) Get LLMs’ response to Instruction 2, i.e., Response 2. Response 2 contains the comment generated by LLMs for the code snippet.

when asking Instruction 2, Instruction 1 and Response 1 are paired as history prompts and answers and input into the LLM.

Critique. Critique prompting improves the quality of LLMs’ answers by asking LLMs to find errors in the answers and correct them. We follow Kim et al. [52] and perform critique prompting on the code summarization task through the six steps below:

- (1) Instruction 1: Similar to zero-shot prompting, input the instruction and the code snippet in the format “Please generate a short comment in one sentence for the following function: $\backslash n \langle code \rangle$ ”
- (2) Get LLMs’ response to Instruction 1, i.e., Response 1. Response 1 contains the temporary comment generated by LLMs for the code snippet.
- (3) Instruction 2: “Review your previous answer and find problems with your answer.”
- (4) Get LLMs’ response to Instruction 2, i.e., Response 2.
- (5) Instruction 3: “Based on the problems you found, improve your answer.”

- (6) Get LLMs’ response to Instruction 3, i.e., Response 3. Response 3 contains the modified comment, which is the final comment of the code snippet.

when prompting each instruction, previous instructions and responses are fed into the LLMs as pairs of history prompts and answers.

Expert. Expert prompting first asks LLMs to generate a description of an expert who can complete the instruction (e.g., through few-shot prompting), and then the description serves as the system prompt for zero-shot prompting. We use the few-shot examples provided by Xu et al. [53] and employ few-shot prompting to let LLMs generate a description of an expert who can “Generate a short comment in one sentence for a function.” This description will replace the default system prompt of LLMs. By default, we use the system prompt [54] of CodeLlama-Instruct for all LLMs to ensure fairness in comparison. Then, we utilize the same steps as zero-shot prompting to adapt LLMs to generate summaries.

D. Experimental Datasets

The sources of the datasets utilized in our experiments include:

CodeSearchNet (CSN). The CodeSearchNet corpus [55] is a vast collection of methods accompanied by their respective comments, written in Go, Java, JavaScript, PHP, Python, and Ruby. This corpus has been widely used in studying code summarization [4], [56], [57]. We use the clean version of the CSN corpus provided by Lu et al. [38] in CodeXGLUE. We randomly select 200 samples for each programming language from the test set of this corpus for experiments.

CCSD. The CCSD dataset is provided by Liu et al. [58]. They crawl data from 300+ projects such as Linux and Redis. The dataset contains 95,281 $\langle function, summary \rangle$ pairs. Similarly, we randomly select 200 samples from the final dataset for experiments.

In addition to the above two sources, we construct three new language datasets to evaluate LLM’s code summarization capabilities across more programming language types.

Erlang, Haskell, and Prolog Datasets. Erlang and Haskell are Functional Programming Languages (FP), and Prolog belongs to Logic Programming Languages (LP). To construct the three datasets, we sort the GitHub repositories whose main language is Erlang/Haskell/Prolog according to the number of stars, and crawl data from the top 50 repositories. Following Husian et al. [55], (1) we remove any projects that do not have a license or whose license does not explicitly permit the re-distribution of parts of the project. (2) We consider the first sentence in the comment as the function summary. (3) We remove data where functions are shorter than three lines or comments containing less than 3 tokens. (4) We remove functions whose names contain the substring “test”. (5) We remove duplicates by comparing the Jaccard similarities of the functions following Allamanis et al. [59]. Finally, we get 7,025/6,759/1,547 pairs of $\langle function, summary \rangle$ pairs. For each language, we randomly select 200 samples for experiments.

All in all, our experiments involve 10 programming languages across 5 types. Note that considering that experiments

TABLE I: Datasets. PP: Procedural Programming Languages, OOP: Object-Oriented Programming Languages, SP: Scripting Programming Languages, FP: Functional Programming Languages, LP: Logic Programming Languages.

Language	Source	Type	Usage
Java	CSN	OOP	RQ1, RQ2, RQ3, RQ4, RQ5
Python	CSN	SP	RQ1, RQ2, RQ3, RQ4
C	CCSD	PP	RQ1, RQ2, RQ3, RQ4
Ruby	CSN	SP	RQ4
PHP	CSN	SP	RQ4
Go	CSN	PP	RQ4
JavaScript	CSN	SP	RQ4
Erlang	by us	FP	RQ4
Haskell	by us	FP	RQ4
Prolog	by us	LP	RQ4

with LLMs are resource-intensive (especially those involving GPT, which are quite costly), not all experiments are conducted on all 10 programming language datasets. Specifically, we first conduct experiments associated with RQ1 and RQ2 on commonly used programming languages, including Java, Python, and C. Analyzing the results of these two RQs helps find a suitable automated evaluation method and a suitable prompting technique. Subsequent experiments for other RQs can be built upon these findings, thereby significantly reducing experimental costs. We use all 10 programming languages in the experiments for RQ4. In the experiments for RQ5, we only use the Java dataset because other programming languages lack readily available comment classifiers. While training such classifiers would be valuable, it falls outside the scope of this paper and is left for future exploration.

IV. RESULTS AND FINDINGS

A. *RQ1: What evaluation methods are suitable for assessing the quality of summaries generated by LLMs?*

1) Experimental Setup.

Comparison Evaluation Methods. Existing automated evaluation methods for code summarization can be divided into the following three categories.

i. *Methods based on summary-summary text similarity* assess the quality of the generated summary by calculating the text similarity between the generated summary and the reference summary. This category of methods is the most widely used in existing code summarization research [4], [14], [17], [19]. The text similarity metrics involved include BLEU, METEOR, and ROUGE-L, which compare the count of n-grams in the generated summary against the reference summary. The scores of BLEU, METEOR, and ROUGE-L are in the range of [0, 1]. The higher the score, the closer the generated summary approximates the reference summary, indicating superior code summarization performance. All scores are computed by the same implementation provided by [36].

ii. *Methods based on summary-summary semantic similarity* evaluate the quality of the generated summary by computing the semantic similarity between the generated summary and the reference summary. Existing research [25] demonstrates that semantic similarity-based methods can effectively alleviate the issues of word overlap-based metrics, where not all

words in a sentence have the same importance and many words have synonyms. In this study, we compare four such methods, including BERTScore [60], SentenceBert with Cosine Similarity (SBCS), SentenceBert with Euclidean Distance (SBED), and Universal Sentence Encoder [61] with Cosine Similarity (USECS). They are commonly used in code summarization studies [25], [62], [63]. BERTScore [60] uses a variant of BERT [64] (we use the default *RoBERTa_{large}*) to embed every token in the summaries, and computes the pairwise inner product between tokens in the reference summary and generated summary. Then it matches every token in the reference summary and the generated summary to compute the precision, recall, and F_1 measure. In our experiment, we report the F_1 measure of BERTScore. The other three methods use a pre-trained sentence encoder (SentenceBert [65] or Universal Sentence Encoder [61]) to produce vector representations of two summary sentences, and then compute the cosine similarity or euclidean distance of the vector representations. SBCS, SBED, and USECS range within [-1,1]. Higher values of SBCS and USECS represent greater similarity, while lower values of SBED indicate greater similarity.

iii. *Methods based on summary-code semantic similarity* assess the quality of the generated summary by computing the semantic similarity between the generated summary and the code snippet to be summarized. Unlike the first two methods, this type of evaluation method does not rely on reference summaries and can effectively avoid issues related to low-quality and outdated reference summaries. SIDE proposed by Mastropaolo et al. [9] is a representative of this type of method. It is based on contrastive learning and has been trained to assess the relevance of a given textual summary for a Java method. Note that it has not been trained on other language datasets. Hence, in our experiment, SIDE is only used to evaluate the Java dataset. SIDE provides a continuous score ranging within [-1,1], where a higher value represents greater similarity. We present the scores reported by the above similarity-based evaluation methods in percentage.

Human Evaluation. We conduct human evaluations as a reference for automated evaluation methods. Comparing the correlation between the results of automated evaluation methods and human evaluation can facilitate achieving the goal of this RQ, which is to find a suitable automated method for assessing the quality of LLM-generated summaries. To do so, we invite 15 volunteers (including 1 PhD candidate, 5 masters, and 9 undergraduates) with more than 3 years of software development experience and excellent English ability to carry out the evaluation. For each sample, we provide volunteers with the code snippet, the reference summary, and summaries generated by four LLMs, where the reference summary and the summaries generated by four LLMs are mixed and out of order. In other words, for each sample, volunteers do not know whether it is a reference or a summary generated by a certain LLM. We follow Shi et al. [8] and ask volunteers to rate the summaries from 1 to 5 based on their quality where a higher score represents a higher quality. The final score of the summaries is the average of scores rated by 15 volunteers.



Here is a piece of code with corresponding comments. Please rate each comment on a scale from 1 to 5, where a higher score indicates better quality. A good comment should: 1) accurately summarize the function of the code; 2) be expressed naturally and concisely, without burdening the developer with reading; 3) help the developer understand the code quickly. Your answer should be in the format 'Comment 0/1/2/3/4: {your rating}'.

Code:
`<code>`
 Comment 0: `<comment 0>`
 Comment 1: `<comment 1>`
 Comment 2: `<comment 2>`



<LLM Response>

Fig. 2: An example of using an LLM as an evaluator.

LLM-based evaluation methods. Inspired by recent work in NLP [26]–[28], we also investigate the feasibility of employing LLMs as evaluators. Its advantage is that it does not rely on the quality of reference summaries, and the evaluation steps can be the same as human evaluation. Specifically, similar to human evaluation, when using LLMs as evaluators, for each sample, we input the code snippet to be summarized, the reference summary, and LLM-generated summaries, and ask LLMs to rate each summary from 1 to 5 where a higher score represents a higher quality of the summary. The specific prompt when using LLMs as evaluators is shown in Figure 2.

Datasets and Prompting Techniques. In this RQ, to reduce the workload of human evaluation volunteers, we randomly select 50 samples from the Java, Python, and C datasets, respectively, which means 150 samples in total. We employ few-shot prompting to adapt the four LLMs to generate summaries for code snippets as recent studies [14], [17], [19] have demonstrated the effectiveness of this prompting technique on code summarization tasks.

2) Experimental Results.

Human Evaluation Results. Table II shows the human evaluation scores for reference summaries and summaries generated by the four LLMs. Observe that the scores of reference summaries in the three datasets are between 3 and 3.5 points, suggesting that the quality of the reference summaries is not very high. Therefore, evaluation methods based on summary-summary similarity may not accurately assess the quality of LLM-generated summaries.

Among the four LLMs, GPT-4 has the highest scores on the Java and C datasets, and GPT-3.5 attains the highest score on the Python dataset. This suggests that the quality of summaries generated by GPT-3.5 and GPT-4 is relatively high.

Finding ► According to human evaluation, the quality of reference summaries in the existing datasets is not particularly high. Summaries from general-purpose LLMs (e.g., GPT-3.5) excel over those from specialized code LLMs (e.g., CodeLlama-Instruct) in quality. ◀

Automated Evaluation Results. Table III displays the scores of the LLM-generated summaries reported by three categories of automated evaluation methods, and LLM-based evaluation methods. Observed that among the three methods based on summary-summary text similarity, 1) the BLEU-based

TABLE II: Human evaluation scores for reference and LLM-generated summaries. The value in parentheses represents the percentage increase or decrease relative to the score of the corresponding reference summary.

Summary from	Human Evaluation Score		
	Java	Python	C
Reference	3.19	3.56	3.05
CodeLlama-Instruct	3.93 (+23.20%)	3.88 (+8.99%)	4.15 (+36.07%)
StarChat- β	3.18 (-0.31%)	3.14 (-11.80%)	3.49 (+14.43%)
GPT-3.5	4.00 (+25.39%)	4.16 (+16.85%)	4.06 (+33.11%)
GPT-4	4.17 (+30.72%)	4.06 (+14.04%)	4.25 (+39.34%)

and ROUGE-L-based methods give StarChat- β the highest scores on all three datasets; 2) the METEOR-based method gives StarChat- β the highest score (i.e., 18.19) on the Java dataset, while gives CodeLlama-Instruct the highest scores (i.e., 21.64 and 17.29) on the Python and C datasets. Among the four methods based on summary-summary semantic similarity, BERTScore, SBCS, and SBED give the best scores to StarChat- β , and USECS gives the best score of 50.69 to CodeLlama-Instruct on the Java dataset. On the Python and C datasets, the four methods consistently give the best scores to CodeLlama-Instruct and StarChat- β , respectively. The summary-code semantic similarity-based method SIDE gives the highest score (i.e., 80.46) to StarChat- β on the Java dataset. On the Java dataset, the five LLM-based methods consistently give the highest scores to GPT-4. While on the Python dataset, they consistently award the highest scores to GPT-3.5. On the C dataset, StarChat- β gives the highest score to CodeLlama-Instruct while other LLMs give the highest score to GPT-4.

Finding ► According to automated evaluation, overall, methods based on summary-summary text/semantic similarity tend to give higher scores to specialized code LLMs StarChat- β and CodeLlama-Instruct, while LLM-based evaluators tend to give higher scores to general-purpose LLMs GPT-3.5 and GPT-4. The summary-code semantic similarity-based method tends to give higher scores to StarChat- β on the Java dataset. ◀

Correlation between Automated Evaluation and Human Evaluation. From Table III, it can be observed that the average scores of reference summaries evaluated by the four LLM-based methods are mostly below 3 points. It means that similar to human evaluation, LLM-based evaluation methods also believe that the quality of the reference summaries is not very high. Besides, LLM-based evaluation methods are inclined to give higher scores to general-purpose LLMs GPT-3.5 and GPT-4, which is the same as human evaluation.

Based on the above observations, we can reasonably speculate that compared to methods based on summary-summary text/semantic similarity and summary-code semantic similarity, LLM-based evaluation methods may be more suitable for evaluating the quality of summaries generated by LLMs. Therefore, we follow [8], [62] and calculate Spearman’s

TABLE III: Automated evaluation scores for reference and LLM-generated summaries. S-S Tex.Sim.: methods based on summary-summary text similarity; S-S Sem.Sim.: methods based on summary-summary semantic similarity; S-C Sem.Sim.: methods based on summary-code semantic similarity. CodeLlama-I: CodeLlama-Instruct. We bold the best score in each column.

Language	Summary from	S-S Tex.Sim.			S-S Sem.Sim.				S-C Sem.Sim.	LLM-based Evaluation Method					Human
		BLEU	METEOR	ROUGE-L	BERTScore	SBCS	SBED	USECS	SIDE	CodeLlama-I (7B)	CodeLlama-I (70B)	StarChat- β	GPT-3.5	GPT-4	
Java	Reference	/	/	/	/	/	/	/	86.15	1.42	2.62	2.58	3.08	2.8	3.19
	CodeLlama-I	13.00	17.90	32.21	87.94	59.61	86.88	50.69	46.62	2.32	3.72	2.80	3.28	3.64	3.93
	StarChat- β	18.95	18.19	38.43	88.69	61.97	83.45	50.57	80.46	2.24	2.52	1.94	2.42	2.50	3.18
	GPT-3.5	12.49	16.74	31.87	87.73	59.47	88.11	48.87	62.04	2.40	3.38	2.40	3.72	3.82	4.00
	GPT-4	9.46	17.02	28.36	86.72	58.83	89.27	46.50	36.12	2.44	3.88	2.60	4.10	4.50	4.17
Python	Reference	/	/	/	/	/	/	/	/	1.48	3.13	2.74	2.84	2.98	3.56
	CodeLlama-I	16.04	21.64	37.80	89.06	61.57	85.40	55.86	/	1.62	3.50	2.60	3.44	3.72	3.88
	StarChat- β	18.35	17.62	37.96	88.92	58.97	87.39	51.54	/	1.94	2.36	1.96	2.40	2.42	3.14
	GPT-3.5	11.95	19.14	30.20	87.63	61.37	86.36	49.54	/	1.96	4.10	2.72	4.32	4.30	4.16
	GPT-4	14.07	20.87	35.38	88.11	60.65	87.04	51.21	/	1.76	3.42	2.54	3.92	4.16	4.06
C	Reference	/	/	/	/	/	/	/	/	1.56	2.25	2.80	2.24	2.62	3.05
	CodeLlama-I	10.92	17.29	28.71	86.38	51.55	95.94	37.95	/	2.62	3.61	3.02	3.82	3.84	4.15
	StarChat- β	15.58	15.57	32.84	87.27	54.85	91.92	40.60	/	2.76	2.64	2.74	2.20	2.62	3.49
	GPT-3.5	12.06	16.00	29.81	86.65	53.61	93.71	39.75	/	3.04	3.38	2.86	3.48	3.66	4.06
	GPT-4	10.07	16.18	28.63	86.03	53.00	94.77	37.30	/	3.18	3.72	2.86	4.00	4.36	4.25

correlation coefficient ρ with the p -value between the results of each automated evaluation method and human evaluation, providing more convincing evidence for this speculation. The Spearman’s correlation coefficient $\rho \in [-1, 1]$ is suitable for judging the correlation between two sequences of discrete ordinal/continuous data, with a higher value representing a stronger correlation [66]. $-1 \leq \rho < 0$, $\rho = 0$, and $0 < \rho \leq 1$ respectively indicate the presence of negative correlation, no correlation, and positive correlation [67]. The p -value helps determine whether the observed correlation is statistically significant or simply due to random chance. By comparing the p -value to a predefined significance level (typically 0.05), we can decide whether to reject the null hypothesis and conclude that the correlation is statistically significant. Due to the page limit, we present the statistical results of ρ and p -value in [29]. The results demonstrate that among all automated evaluation methods, there is a significant positive correlation between the GPT-4-based evaluation method and human evaluation in scoring the quality of summaries generated by most LLMs, followed by the GPT-3.5-based evaluation method. For other automated evaluation methods, in most cases, their correlation with human evaluation is negative or weakly positive. In a shot, existing metrics are inadequate for LLM-generated summaries. The main reason for this phenomenon lies in the low quality of reference summaries. The reference summaries are short and detail-lacking, while LLMs can generate detailed useful summaries not fully covered by the reference summaries. Therefore, the similarity with the reference summary cannot accurately reflect the quality of the LLM-generated summaries. In addition, BLEU penalizes long summaries, which also affects the correlation between BLEU and human evaluation. Although SIDE does not rely on reference summaries, it is trained on reference summaries and is therefore also affected. Re-training SIDE on LLM-generated summaries requires high-quality and extensive training data, while collecting large-scale LLM-generated summaries, especially from LLMs like GPT-4, is costly. Numerous factors influence LLM-generated summary quality, complicating the collection of high-quality

```

1 private void setPeople(final Email emailWithData, final MimeMessage msgToSet) throws MessagingException {
2     msgToSet.setFrom(emailWithData.from().toInternetAddress());
3     msgToSet.setReplyTo(EmailAddress.convert(emailWithData.replyTo()));
4     setRecipients(emailWithData, msgToSet);
5 }

```

(a) A Java code snippet c_1

Summary from	Score	Summary
Reference	2.8	Sets FROM REPLY - TO and recipients.
CodeLlama-I	3.8	Set the "from" and "reply-to" fields of the email message.
StarChat- β	3.4	Sets the To, Cc, and Bcc recipients for an email .
GPT-3.5	4.0	Sets the sender, reply-to, and recipients of the email message.
GPT-4	4.4	Sets the sender, reply-to address, and recipients on a MimeMessage using data from an Email object.

(b) Reference summary and LLM-generated summaries for c_1

Fig. 3: An example of code, the reference summary, and LLM-generated summaries

data. Based on the above observations, we draw the conclusion that compared with other automated evaluation methods, the GPT-4-based method is more suitable for evaluating the quality of summaries generated by LLMs. In the subsequent RQs, we uniformly employ the GPT-4-based method to assess the quality of LLM-generated summaries. We set the temperature to default value 1 when using GPT-4 as the evaluator.

Summary ▶ Among all automated evaluation methods, the GPT-4-based method overall has the strongest correlation with human evaluation. Therefore, it is recommended to adopt the GPT-4-based method to evaluate the quality of LLM-generated summaries. ◀

Why do LLMs give such a score?

To investigate the scoring criteria of LLMs, we ask them to give explanations for their evaluation scores. The prompt we use is “Here is a piece of code with corresponding comments and your previous rate for each comment on a scale from 1 to 5, where a higher score indicates better quality. Please explain why you scored this way.”

Figure 4 shows LLMs’ explanations of evaluation scores for the reference summary and generated summaries of the example in Figure 3. Due to the page limit, we only show the

Reference	3	This comment is concise and provides a brief summary of the code's purpose. However, it does not accurately describe the code's functionality, as it does not mention the "recipients" part.
CodeLlama-I	2	This comment is not accurate and does not provide a clear description of the code's purpose. It does not mention the "recipients" part, which is also set by the code.
StarChat- β	2	This comment is not accurate and does not provide a clear description of the code's purpose. It does not mention the "from" and "reply-to" fields, which are also set by the code.
GPT-3.5	2	This comment is not accurate and does not provide a clear description of the code's purpose. It does not mention the "from" and "reply-to" fields, which are also set by the code.
GPT-4	4	This comment is accurate and provides a clear description of the code's purpose. It mentions all the important aspects of the code's functionality.

(a) CodeLlama-Instruct (70B)'s explanations

Reference	3	This comment is not very descriptive and does not provide any context about what the code is doing. It is also not clear what the "FROM REPLY - TO" part refers to.
CodeLlama-I	4	This comment provides a clear description of what the code is doing and provides some context. However, it could be more specific and include information about the type of email message that is being handled.
StarChat- β	3	This comment is not very descriptive and does not provide any context about what the code is doing. It is also not clear what the "To, Cc, and Bcc" part refers to.
GPT-3.5	4	This comment provides a clear and concise description of what the code is doing. However, it could be more specific and include information about the type of email message that is being handled.
GPT-4	4	This comment provides a clear description of what the code is doing and provides some context. However, it could be more specific and include information about the Email object that is being used.

(b) StarChat- β 's explanations

Reference	2	This comment is not very clear and seems to have some typos or missing words . It could be improved by providing more context and clarifying the purpose of the code.
CodeLlama-I	1	This comment is not accurate as it only mentions setting the "from" and "reply-to" fields, while the code also sets other recipients. It could be improved by providing a more comprehensive description of the method's functionality.
StarChat- β	3	This comment is clear about what the code does but lacks information about setting the sender and reply-to fields. It could be improved by mentioning all recipients being set in the method.
GPT-3.5	4	This comment is clear and provides a good overview of the functionality of the code. It could be slightly improved by specifying that it is specifically for an email message.
GPT-4	5	This comment is clear, concise , and accurately describes the purpose of the code. It provides relevant information about what the method does and how it relates to the Email object.

(c) GPT-3.5's explanations

Reference	3	This comment is somewhat vague and lacks proper formatting and clarity (e.g., "FROM REPLY - TO" is not a standard way to refer to email fields). It does not mention that the method is setting these fields based on an 'Email' object, not does it specify that it's for a 'MimeMessage'. However, it does correctly identify the three main actions of the method, hence the mid-range score.
CodeLlama-I	2	This comment correctly identifies two of the three actions performed by the method (setting the 'from' and 'reply-to' fields), but it omits the fact that the method also sets recipients. Additionally, it does not specify that the settings are for a 'MimeMessage' or that they are based on an 'Email' object. The omission and lack of detail result in a lower rating.
StarChat- β	1	This comment is misleading because it incorrectly states the method's functionality. The method sets the "from" and "reply-to" fields, not the "To, Cc, and Bcc" recipients. This inaccuracy justifies the lowest rating.
GPT-3.5	4	This comment is concise and accurately describes the main actions of the method. However, it does not mention that the settings are for a 'MimeMessage' or that they are based on an 'Email' object, which makes it slightly less informative than Comment 1, resulting in a slightly lower rating.
GPT-4	5	This comment is clear, informative , and accurately describes the method's functionality. It specifies that the method sets the sender, reply-to, and recipients, and it also clarifies that these are being set for a 'MimeMessage' based on a provided 'Email' object. This level of detail and clarity justifies the highest rating.

(d) GPT-4's explanations

Fig. 4: LLMs' explanations of evaluation scores for the reference summary and LLM-generated summaries of c_1

results of CodeLlama-Instruct (70B), which performs better, instead of CodeLlama-Instruct (7B). Observe that LLM's scoring criteria include accuracy, clarity, informativeness, and conciseness. Accuracy focuses on whether there is misleading information. Clarity focuses on whether the expression of a sentence is easy to understand. For example, the reference summary is evaluated by GPT-4 as not clear because "'FROM REPLY - TO' is not a standard way to refer to email fields". Informativeness focuses on whether the summary contains enough details. As for conciseness, since the summaries generated by GPT-3.5 and GPT-4 are also evaluated as concise, we can infer that LLMs' definition of concise is not equal to being as brief as the reference summary and the LLM-generated summaries are also considered concise. Therefore, the main factors affecting LLMs' scoring are accuracy, clarity,

TABLE IV: Effectiveness of different prompting techniques

Model	Prompting Technique	Java	Python	C
CodeLlama-Instruct	zero-shot	3.42	2.98	3.41
	few-shot	3.78	3.75	3.91
	chain-of-thought	3.21	3.14	3.37
	critique	2.15	2.02	2.13
	expert	3.13	3.35	1.70
StarChat- β	zero-shot	2.71	2.85	2.86
	few-shot	2.60	2.37	2.68
	chain-of-thought	2.86	2.77	3.06
	critique	2.36	2.57	2.60
	expert	2.66	3.02	3.01
GPT-3.5	zero-shot	3.90	3.96	3.93
	few-shot	3.56	3.97	3.56
	chain-of-thought	3.36	3.47	3.36
	critique	3.09	3.21	3.31
	expert	2.72	3.43	3.49
GPT-4	zero-shot	4.50	4.55	4.42
	few-shot	4.66	4.16	4.18
	chain-of-thought	4.57	4.60	4.44
	critique	4.41	4.44	4.34
	expert	4.52	4.23	4.50

and informativeness. That is, whether there is misleading information, whether the expression is easy to understand, and whether the summary contains enough details. This also explains why LLMs do not give high scores to reference summaries. Reference summaries are generally short, do not cover every detail in the code snippet, and may contain terms in the code snippet which increases the difficulty of understanding. However, although the scoring criteria of LLMs are similar, the scoring results are still different. This is because LLMs' capabilities to understand code and summaries are different. As the SOTA LLM, GPT-4 achieves the best performance. It can accurately identify whether each summary contains all the details, while other LLMs may miss some of the details according to their explanation. In addition, GPT-4 does a better job in judging whether the summary is clear, as it points out that the phrase "To, Cc, and Bcc" is not easy to understand while GPT-3.5 and CodeLlama-Instruct (70B) does not point out this problem.

Finding ► The main factors affecting LLMs' scoring are accuracy, clarity, and informativeness. GPT-4 evaluation results have stronger correlation to human evaluation than other LLMs owing to its better understanding of code and summaries. ◀

B. RQ2: How effective are different prompting techniques in adapting LLMs to the code summarization task?

1) *Experimental Setup*. The experimental dataset comprises 600 samples from Java, Python, and C datasets collectively. 2) *Experimental Results*. Table IV presents the scores reported by the GPT-4 evaluation method for summaries generated by four LLMs using five prompting techniques. Observe that when the base model is CodeLlama-Instruct, few-shot prompting consistently performs best on all three datasets. When the base model is StarChat- β , chain-of-thought prompting performs best on all the Java and C datasets, while expert prompting excels on the Python dataset. When selecting GPT-3.5 as the base model, the simplest zero-shot prompting

surprisingly achieves the highest scores on the Java and C datasets, and is only slightly worse than few-shot prompting on the Python dataset. When using GPT-4 as the base model, chain-of-thought prompting overall performs best.

For the specific LLM and programming language, there is no guarantee that intuitively more advanced prompting techniques will surpass simple zero-shot prompting. For example, on the Java dataset, when selecting any of StarChat- β , GPT-3.5, and GPT-4 as the base model, few-shot prompting yields lower scores than zero-shot prompting. Contrary to the findings of previous studies [14], [17], the GPT-4-based evaluation method does not consider that few-shot prompting will improve the quality of generated summaries. This discrepancy may arise because previous studies evaluated the quality of LLM-generated summaries using BLEU, METEOR, and ROUGE-L, which primarily assess text/semantic similarity with reference summaries. However, as we mentioned in Section IV-A, reference summaries contain low-quality noisy data that undermines their reliability. Therefore, achieving greater similarity with reference summaries does not necessarily imply that the human/GPT-4-based evaluation method will perceive the summary to be of higher quality.

 **Summary** ► The more advanced prompting techniques expected to perform better may not necessarily outperform simple zero-shot prompting. In practice, selecting the appropriate prompting technique requires considering the base LLM and the programming language. ◀

C. RQ3: How do different model settings affect LLMs' code summarization performance?

1) *Experimental Setup.* There are three key model settings/parameters, including top_k, top_p, and temperature, that allow the user to control the randomness of text (code summary in our scenario) generated by LLMs. Considering that GPT-3.5 and GPT-4 do not support the top_k setting, we only conduct experiments with the top_p and temperature.

Top_p: In each round of token generation, LLMs sort tokens by probability from high to low and keep tokens whose probability adds up to (no more than) top_p. For example, top_p = 0.1 means only the tokens comprising the top 10% probability mass are considered. The larger the top_p is, the more tokens are sampled. Thus tokens with low probabilities have a greater chance of being selected, so the summary generated by LLMs is more random.

Temperature: Temperature adjusts the probability of tokens after top_p sampling. The higher the temperature, the less the difference between the adjusted token probabilities. Therefore, the token with a low probability has a greater chance of being selected, so the generated summary is more random. If the temperature is set to 0, the generated summary is the same every time.

Top_p and temperature are alternatives and one should only modify one of the two parameters at a time [68]. Therefore, the questions we want to answer are: (1) Does top_p/temperature impact the quality of LLM-generated summaries? (2) As alter-

TABLE V: Influence of different model settings. We bold the scores of the best setting combinations on each dataset.

Model	Top_p	Temperature	Java	Python	C
CodeLlama-Instruct	0.5	0.1	3.81	3.83	4.10
		0.5	3.72	3.85	4.08
		1.0	3.91	3.81	4.11
	0.75	0.1	3.76	3.87	4.02
		0.5	3.91	3.73	4.01
		1.0	3.80	3.79	3.88
	1.0	0.1	3.78	3.75	3.91
		0.5	3.91	3.75	3.99
		1.0	3.73	3.59	3.60
StarChat- β	0.5	0.1	2.49	2.42	2.72
		0.5	2.47	2.36	2.70
		1.0	2.49	2.29	2.75
	0.75	0.1	2.50	2.35	2.66
		0.5	2.45	2.47	2.80
		1.0	2.48	2.37	2.71
	1.0	0.1	2.60	2.37	2.68
		0.5	2.53	2.45	2.77
		1.0	2.54	2.38	2.69
GPT-3.5	0.5	0.1	3.41	3.60	3.40
		0.5	3.45	3.73	3.38
		1.0	3.52	3.68	3.42
	0.75	0.1	3.54	3.66	3.35
		0.5	3.55	3.65	3.24
		1.0	3.46	3.64	3.41
	1.0	0.1	3.56	3.97	3.56
		0.5	3.55	3.71	3.42
		1.0	3.41	3.72	3.52
GPT-4	0.5	0.1	4.44	4.25	4.33
		0.5	4.47	4.30	4.31
		1.0	4.45	4.31	4.29
	0.75	0.1	4.48	4.27	4.31
		0.5	4.46	4.34	4.26
		1.0	4.47	4.33	4.36
	1.0	0.1	4.66	4.16	4.18
		0.5	4.43	4.27	4.33
		1.0	4.40	4.18	4.33

native parameters that both control the randomness of LLMs, do top_p and temperature have a difference in the degree of influence on the quality of LLM-generated summaries?

Drawing from a review of related work (see Section II), we find that existing LLM-based code summarization studies pay more attention to few-shot prompting. Since no prompting technique outperforms others on all LLMs, we uniformly employ few-shot prompting in RQ3, RQ4, and RQ5 to facilitate comparing our findings with prior studies.

2) *Experimental Results.* TABLE V shows the scores evaluated by the GPT-4 evaluation method for the summaries generated by LLMs under different top_p and temperature settings. It is observed that the impact of top_p and temperature on the quality of LLM-generated summaries is specific to the base LLM and programming language. For example, when top_p=0.5, as temperature increases, the quality of GPT-4-generated summaries for Python code snippets increases, while those for C code snippets decrease. Another example is that when top_p=0.5, as the temperature rises, the quality of GPT-4-generated Java comments first increases and then decreases, whereas CodeLlama-Instruct is exactly the opposite, first decreases and then increases. Regarding the difference in

TABLE VI: Effectiveness of LLMs in summarizing code snippets written in different types of programming languages. CodeLlama-I: CodeLlama-Instruct.

Model	OOP		PP		SP			FP		LP
	Java	C	Go	Python	Ruby	PHP	JavaScript	Erlang	Haskell	Prolog
CodeLlama-I	3.78	3.91	3.86	3.75	3.98	3.88	4.03	3.51	3.58	3.23
StarChat- β	2.60	2.68	2.97	2.37	2.79	2.73	2.67	2.68	2.88	2.34
GPT-3.5	3.56	3.56	4.14	3.97	3.64	3.99	3.53	3.57	3.44	3.42
GPT-4	4.66	4.18	4.36	4.16	4.37	4.31	4.29	4.23	4.22	4.05

influence between top_p and temperature, it is observed that in most cases the influence of the two parameters is similar. For example, for C code snippets, when one parameter (top_p or temperature) is fixed, as the other parameter (temperature or top_p) grows, the quality of GPT-3.5-generated summaries first decreases and then increases.

Summary ▶ The impact of top_p and temperature on the quality of generated summaries is specific to the base LLM and programming language. As alternative parameters, top_p and temperature have similar influence on the quality of LLM-generated summaries. The impact of top_p and temperature on GPT-4 is small. ◀

D. RQ4: How do LLMs perform in summarizing code snippets written in different types of programming languages?

1) *Experimental Setup*. We conduct experiments on all 10 programming language datasets. As in RQ3, we uniformly employ few-shot prompting to adapt LLMs.

2) *Experimental Results*. Table VI shows the performance evaluated by the GPT-4 evaluation method for the four LLMs on five types of programming languages. It is observed that for OOP (i.e., Java), GPT-4 performs best, followed by CodeLlama-Instruct, GPT-3.5, and StarChat- β . For PP, GPT-4 performs best on both C and Go, while StarChat- β performs worst on both. The smallest LLM CodeLlama-Instruct outperforms GPT-3.5 on C (3.91 vs. 3.56), but vice versa on Go (3.86 vs. 4.14). Additionally, except for CodeLlama-Instruct, which performs slightly worse on Go than on C (3.86 vs. 3.91), the other three LLMs perform better on Go than on C. For SP, GPT-4 consistently performs best on all four languages. Surprisingly, CodeLlama-Instruct outperforms GPT-3.5 on both Ruby and JavaScript. All four LLMs perform better on PHP than on Python. For FP, the performance of two specialized code LLMs (i.e., CodeLlama-Instruct and StarChat- β) is better on Haskell than on Erlang, while the opposite is true for the two general-purpose LLMs (i.e., GPT-3.5 and GPT-4). For LP, GPT-4 still performs best, followed by GPT-3.5, CodeLlama-Instruct, and StarChat- β . Across all five types of languages, the four LLMs consistently perform the worst on LP, which indicates that summarizing logic programming language code is the most challenging. One possible reason is that fewer Prolog datasets are available for training these LLMs compared to other programming languages. The scale of the Prolog dataset we collected can support this reason.

TABLE VII: Statistics of six sub-datasets divided from the CSN-Java test dataset according to comment intention

Summary Category	Number of Samples	Sample Ratio
What	6,132	0.56
Why	1,190	0.11
How-it-is-done	2,242	0.20
Property	1,174	0.11
How-to-use	180	0.02
Others	37	< 0.01

Summary ▶ GPT-4 surpasses the other three LLMs on all five types of programming languages. For PP, LLMs overall perform better on Go than on C. For SP, all four LLMs perform better on PHP than on Python. For FP, specialized code LLMs (e.g., StarChat- β) perform better on Haskell than on Erlang, whereas the reverse is true for general-purpose LLMs (e.g., GPT-4). All four LLMs perform worse in summarizing LP code snippets. ◀

E. RQ5: How do LLMs perform on different categories of summaries?

1) *Experimental Setup*. Following [3], [43], [44], we classify code summaries into the following six categories.

What: describes the functionality of the code snippet. It helps developers to understand the main functionality of the code without diving into implementation details. An example is “Pushes an item onto the top of this stack”.

Why: explains the reason why the code snippet is written or the design rationale of the code snippet. It is useful when methods’ objective is masked by complex implementation. An application scenario of Why summaries is to explain the design rationale of overloaded functions.

How-it-is-done: describes the implementation details of the code snippet. Such information is critical for developers to understand the subject, especially when the code complexity is high. For instance, “Shifts any subsequent elements to the left.” is a How-it-is-done comment.

Property: asserts properties of the code snippet, e.g., function’s pre-conditions/post-conditions. “This method is not a constant-time operation.” is a Property summary.

How-to-use: describes the expected set-up of using the code snippet, such as platforms and compatible versions. For example, “This method can be called only once per call to next()” is a How-to-use summary.

Others: Comments that do not fall into the above five categories are classified as Others summaries, such as “The implementation is awesome.”. Following Mu et al. [44], we consider the $\langle \text{code}, \text{summary} \rangle$ pairs with Others comments as noisy data, and remove them if identified.

We employ the comment classifier COIN provided by Mu et al. [44] to classify the CSN-Java dataset according to the comment intention type. The test dataset is divided into six sub-datasets, as shown in Table VII. To facilitate comparison between different categories, we randomly select 180 samples from each sub-dataset. As in RQ4, we uniformly employ few-shot prompting to adapt LLMs. For each sub-dataset with

TABLE VIII: Effectiveness of LLMs in generating different categories of summaries

Model	What	Why	How-it-is-done	Property	How-to-use
CodeLlama-Instruct	4.15	4.29	3.85	4.19	3.96
StarChat- β	2.68	2.78	2.77	2.94	2.52
GPT-3.5	3.61	3.54	3.97	3.54	4.17
GPT-4	4.40	4.28	4.31	4.06	4.22

different intention types, the few-shot example is of the same intention type from the training dataset.

2) *Experimental Results.* Table VIII presents the results evaluated by the GPT-4 evaluation method for the four LLMs in generating five categories of summaries. Observe that CodeLlama-Instruct performs worse in generating How-it-is-done summaries than generating the other four categories of summaries. StarChat- β gets the lowest score of 2.52 in generating How-to-use summaries. Both GPT-3.5 and GPT-4 are not as good at generating Property summaries compared to generating other categories of summaries. Surprisingly, the smallest LLM CodeLlama-Instruct slightly outperforms the advanced GPT-4 in generating Why (4.29 vs. 4.28) and Property (4.19 vs. 4.06) summaries. Additionally, compared with GPT-3.5, CodeLlama-Instruct achieves higher scores in generating What, Why, and Property summaries. Certainly, it is undeniable that the reason for this phenomenon is that the optimal prompting technique for GPT-3.5 and GPT-4 is not few-shot prompting. This phenomenon is also exciting because it implies that most ordinary developers or teams who lack sufficient resources (e.g., GPUs) have the opportunity to utilize open-source and small-scale LLMs to achieve code summarization capabilities close to (or even surpass) those of commercial gigantic LLMs.

 **Summary** ► The four LLMs excel in generating different categories of summaries. The smallest CodeLlama-Instruct slightly outperforms the advanced GPT-4 in generating Why and Property summaries. StarChat- β is not proficient at generating How-to-use summaries. GPT-3.5 and GPT-4 perform worse in generating Property summaries than other categories of summaries. ◀

V. THREATS TO VALIDITY

Our empirical study may contain several threats to validity that we have attempted to relieve.

Threats to External Validity. The threats to external validity lie in the generalizability of our findings. One threat to the validity of our study is that LLMs usually generate varied responses for identical input across multiple requests due to their inherent randomness, while conclusions drawn from random results may be misleading. To mitigate this threat, considering that StarChat- β and CodeLlama-Instruct do not support setting the temperature to 0, we uniformly set it to 0.1 to reduce randomness except for RQ3. In RQ2-RQ5, to make the evaluation scores more deterministic, we set the temperature to 0 when using GPT-4 as the evaluator. Addi-

tionally, for other RQs, we conduct experiments on multiple programming languages to support our findings.

Threats to Internal Validity. A major threat to internal validity is the potential mistakes in the implementation of metrics and models. To mitigate this threat, we use the publicly available code from previous studies [9], [36] for BLEU, METEOR, ROUGE-L, and SIDE. For COIN, BERTScore, SentenceBert, Universal Sentence Encoder, StarChat- β [69] and CodeLlama-Instruct [70], and GPT-3.5/GPT-4 [71], we use the script provided along with the model to run.

Another threat lies in the processing of LLM’s responses. Usually, the output of LLMs is a paragraph, not a sentence of code summary (code comment) that we want. The real code summary may be the first sentence in the LLMs’ response, or it may be returned in the comment before the code such as “/** <code summary> */”, etc. Therefore, we designed a series of heuristic rules to extract the code summary. We have made our script for extracting code summaries from LLMs’ responses public for the community to review.

VI. CONCLUSION

In this paper, we provide a comprehensive study covering multiple aspects of code summarization in the era of LLMs. Our interesting and significant findings include, but are not limited to, the following aspects. 1) Compared with existing automated evaluation methods, the GPT-4-based evaluation method is more fitting for assessing the quality of LLM-generated summaries. 2) The advanced prompting techniques anticipated to yield superior performance may not invariably surpass the efficacy of straightforward zero-shot prompting. 3) The two alternative model settings have a similar impact on the quality of LLM-generated summaries, and this impact varies by the base LLM and programming language. 4) LLMs exhibit inferior performance in summarizing LP code snippets. 5) CodeLlama-Instruct with 7B parameters demonstrates superior performance over the advanced GPT-4 in generating Why and Property summaries. Our comprehensive research findings will aid subsequent researchers in swiftly grasping the various facets of LLM-based code summarization, thereby promoting the development of this field.

ACKNOWLEDGMENT

The authors would like to thank the anonymous reviewers for their insightful comments. This work is supported partially by the National Research Foundation, Singapore, and DSO National Laboratories under the AI Singapore Programme (AISG Award No: AISG2-GC-2023-008), the National Research Foundation, Singapore, and the Cyber Security Agency under its National Cybersecurity R&D Programme (NCRP25-P04-TAICeN), and the National Natural Science Foundation of China (61932012, 62372228). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of the National Research Foundation, Singapore and Cyber Security Agency of Singapore. Chunrong Fang is the corresponding author.

REFERENCES

- [1] S. N. Woodfield, H. E. Dunsmore, and V. Y. Shen, "The effect of modularization and comments on program comprehension," in *Proceedings of the 5th International Conference on Software Engineering*. San Diego, California, USA: IEEE Computer Society, March 9-12 1981, pp. 215–223.
- [2] S. C. B. de Souza, N. Anquetil, and K. M. de Oliveira, "A study of the documentation essential to software maintenance," in *Proceedings of the 23rd Annual International Conference on Design of Communication: documenting & Designing for Pervasive Information*. Coventry, UK: ACM, September 21-23 2005, pp. 68–75.
- [3] J. Zhai, X. Xu, Y. Shi, G. Tao, M. Pan, S. Ma, L. Xu, W. Zhang, L. Tan, and X. Zhang, "CPC: Automatically classifying and propagating natural language comments via program analysis," in *Proceedings of the 42nd International Conference on Software Engineering*. Seoul, South Korea: ACM, 27 June - 19 July 2020, pp. 1359–1371.
- [4] W. Sun, C. Fang, Y. Chen, Q. Zhang, G. Tao, Y. You, T. Han, Y. Ge, Y. Hu, B. Luo, and Z. Chen, "An extractive-and-abstractive framework for source code summarization," *ACM Transactions on Software Engineering and Methodology*, vol. Just Accepted, no. 1, pp. 1–39, 2023.
- [5] M. L. Vázquez, B. Li, C. Vendome, and D. Poshvanyk, "How do developers document database usages in source code? (N)," in *Proceedings of the 30th International Conference on Automated Software Engineering*. Lincoln, NE, USA: IEEE Computer Society, November 9-13 2015, pp. 36–41.
- [6] F. Wen, C. Nagy, G. Bavota, and M. Lanza, "A large-scale empirical study on code-comment inconsistencies," in *Proceedings of the 27th International Conference on Program Comprehension*. Montreal, QC, Canada: IEEE / ACM, May 25-31 2019, pp. 53–64.
- [7] X. Hu, X. Xia, D. Lo, Z. Wan, Q. Chen, and T. Zimmermann, "Practitioners' expectations on automated code comment generation," in *Proceedings of the 44th International Conference on Software Engineering*. Pittsburgh, PA, USA: ACM, May 25-27 2022, pp. 1693–1705.
- [8] E. Shi, Y. Wang, L. Du, J. Chen, S. Han, H. Zhang, D. Zhang, and H. Sun, "On the evaluation of neural code summarization," in *Proceedings of the 44th International Conference on Software Engineering*. Pittsburgh, USA: IEEE, May 21–29 2022, pp. 1597–1608.
- [9] A. Mastropaolo, M. Ciniselli, M. Di Penta, and G. Bavota, "Evaluating code summarization techniques: A new metric and an empirical characterization," *arXiv e-prints*, pp. arXiv-2312, 2023.
- [10] W. Sun, C. Fang, Y. You, Y. Chen, Y. Liu, C. Wang, J. Zhang, Q. Zhang, H. Qian, W. Zhao *et al.*, "A prompt learning framework for source code summarization," *arXiv preprint arXiv:2312.16066*, 2023.
- [11] M. Du, F. He, N. Zou, D. Tao, and X. Hu, "Shortcut learning of large language models in natural language understanding," *Communications of the ACM*, vol. 67, no. 1, pp. 110–120, 2023.
- [12] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, and J. M. Zhang, "Large language models for software engineering: Survey and open problems," *arXiv preprint arXiv:2310.03533*, 2023.
- [13] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *arXiv preprint arXiv:2308.10620*, 2023.
- [14] T. Ahmed and P. T. Devanbu, "Few-shot training llms for project-specific code-summarization," in *Proceedings of the 37th International Conference on Automated Software Engineering*. Rochester, MI, USA: ACM, October 10-14 2022, pp. 177:1–177:5.
- [15] C. Wang, Y. Yang, C. Gao, Y. Peng, H. Zhang, and M. R. Lyu, "No more fine-tuning? an experimental evaluation of prompt tuning in code intelligence," in *Proceedings of the 30th Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. Singapore, Singapore: ACM, November 14-18 2022, pp. 382–394.
- [16] W. Sun, C. Fang, Y. You, Y. Miao, Y. Liu, Y. Li, G. Deng, S. Huang, Y. Chen, Q. Zhang, H. Qian, Y. Liu, and Z. Chen, "Automatic code summarization via chatgpt: How far are we?" *CoRR*, vol. abs/2305.12865, pp. 1–13, 2023.
- [17] M. Geng, S. Wang, D. Dong, H. Wang, G. Li, Z. Jin, X. Mao, and X. Liao, "Large language models are few-shot summarizers: Multi-intent comment generation via in-context learning," in *Proceedings of the 46th International Conference on Software Engineering*. Lisbon, Portugal: ACM, April 14-20 2024, pp. 39:1–39:13.
- [18] S. Gao, W. Mao, C. Gao, L. Li, X. Hu, X. Xia, and M. R. Lyu, "Learning in the wild: Towards leveraging unlabeled data for effectively tuning pre-trained code models," in *Proceedings of the 46th International Conference on Software Engineering*. Lisbon, Portugal: ACM, April 14–20 2024, pp. 1–13.
- [19] S. Gao, X. Wen, C. Gao, W. Wang, H. Zhang, and M. R. Lyu, "What makes good in-context demonstrations for code intelligence tasks with llms?" in *Proceedings of the 38th International Conference on Automated Software Engineering*. Luxembourg: IEEE, September 11-15 2023, pp. 761–773.
- [20] H. Wu, H. Zhao, and M. Zhang, "Code summarization with structure-induced transformer," in *Proceedings of the Findings of the 59th Annual Meeting of the Association for Computational Linguistics*. Online Event: Association for Computational Linguistics, August 1-6 2021, pp. 1078–1090.
- [21] X. Hu, G. Li, X. Xia, D. Lo, S. Lu, and Z. Jin, "Summarizing source code with transferred API knowledge," in *Proceedings of the 27th International Joint Conference on Artificial Intelligence*. Stockholm, Sweden: ijcai.org, July 13-19 2018, pp. 2269–2275.
- [22] K. Papineni, S. Roukos, T. Ward, and W. Zhu, "BLEU: A method for automatic evaluation of machine translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*. Philadelphia, PA, USA: ACL, July 6-12 2002, pp. 311–318.
- [23] S. Banerjee and A. Lavie, "METEOR: an automatic metric for MT evaluation with improved correlation with human judgments," in *Proceedings of the Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*. Ann Arbor, Michigan, USA: Association for Computational Linguistics, June 29 2005, pp. 65–72.
- [24] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Proceedings of the 42nd Annual Meeting of the Association for Computational Linguistics – workshop on Text Summarization Branches Out*. Barcelona, Spain: Association for Computational Linguistics, July 21-26 2004, pp. 74–81.
- [25] S. Haque, Z. Eberhart, A. Bansal, and C. McMillan, "Semantic similarity metrics for evaluating source code summarization," in *Proceedings of the 30th International Conference on Program Comprehension*. Virtual Event: ACM, May 16-17 2022, pp. 36–47.
- [26] J. Wang, Y. Liang, F. Meng, H. Shi, Z. Li, J. Xu, J. Qu, and J. Zhou, "Is chatgpt a good NLG evaluator? A preliminary study," *CoRR*, vol. abs/2303.04048, no. 1, pp. 1–11, 2023.
- [27] I. Vykopal, M. Pikuliak, I. Srba, R. Moro, D. Macko, and M. Bielikova, "Disinformation capabilities of large language models," *arXiv preprint arXiv:2311.08838*, 2023.
- [28] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, "G-eval: NLG evaluation using gpt-4 with better human alignment," in *Proceedings of the 28th Conference on Empirical Methods in Natural Language Processing*. Singapore: Association for Computational Linguistics, December 6-10 2023, pp. 2511–2522.
- [29] W. Sun, Y. Miao, Y. Li, H. Zhang, C. Fang, Y. Liu, G. Deng, Y. Liu, and Z. Chen, "Artifacts of this study," site: <https://github.com/wssun/LLM4CodeSummarization>, 24, accessed: 2024-08-07.
- [30] X. Hu, G. Li, X. Xia, D. Lo, and Z. Jin, "Deep code comment generation with hybrid lexical and syntactical information," *Empirical Software Engineering*, vol. 25, no. 3, pp. 2179–2217, 2020.
- [31] S. Haiduc, J. Aponte, and A. Marcus, "Supporting program comprehension with source code summarization," in *Proceedings of the 32nd International Conference on Software Engineering*. Cape Town, South Africa: ACM, 1-8 May 2010, pp. 223–226.
- [32] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *Proceedings of the 3rd International Conference on Learning Representations*. San Diego, CA, USA: OpenReview.net, May 7-9 2015, pp. 1–15.
- [33] K. Cho, B. van Merriënboer, D. Bahdanau, and Y. Bengio, "On the properties of neural machine translation: Encoder-decoder approaches," in *Proceedings of Eighth Workshop on Syntax, Semantics and Structure in Statistical Translation*. Doha, Qatar: Association for Computational Linguistics, 25 October 2014, pp. 103–111.
- [34] W. U. Ahmad, S. Chakraborty, B. Ray, and K. Chang, "A transformer-based approach for source code summarization," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Online: Association for Computational Linguistics, July 5-10 2020, pp. 4998–5007.

- [35] D. Gros, H. Sezhiyan, P. Devanbu, and Z. Yu, "Code to comment translation: data, metrics, baselining & evaluation," in *Proceedings of the 35th International Conference on Automated Software Engineering*. Melbourne, Australia: IEEE, September 21-25 2020, pp. 746-757.
- [36] J. Zhang, X. Wang, H. Zhang, H. Sun, and X. Liu, "Retrieval-based neural source code summarization," in *Proceedings of the 42nd International Conference on Software Engineering*. Seoul, South Korea: ACM, 27 June - 19 July 2020, pp. 1385-1397.
- [37] D. Fried, A. Aghajanyan, J. Lin, S. Wang, E. Wallace, F. Shi, R. Zhong, S. Yih, L. Zettlemoyer, and M. Lewis, "Incoder: A generative model for code infilling and synthesis," in *Proceedings of the 11th International Conference on Learning Representations*. Kigali, Rwanda: OpenReview.net, May 1-5 2023, pp. 1-14.
- [38] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. B. Clement, D. Drain, D. Jiang, D. Tang, G. Li, L. Zhou, L. Shou, L. Zhou, M. Tufano, M. Gong, M. Zhou, N. Duan, N. Sundaresan, S. K. Deng, S. Fu, and S. Liu, "Codexglue: A machine learning benchmark dataset for code understanding and generation," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, virtual, December 2021, pp. 1-14.
- [39] C. Su and C. McMillan, "Distilled GPT for source code summarization," *Automated Software Engineering*, vol. 31, no. 1, p. 22, 2024.
- [40] T. A. and Kunal Suresh Pai, P. Devanbu, and E. T. Barr, "Automatic semantic augmentation of language model prompts (for code summarization)," in *Proceedings of the 46th International Conference on Software Engineering*. Lisbon, Portugal: ACM, April 14-20 2024, pp. 1-13.
- [41] S. A. Rukmono, L. Ochoa, and M. R. Chaudron, "Achieving high-level software component summarization via hierarchical chain-of-thought prompting and static code analysis," in *Proceedings of the 2023 International Conference on Data and Software Engineering*. Toba, Indonesia: IEEE, September 07-08 2023, pp. 7-12.
- [42] Y. Choi and J. Lee, "Codeprompt: Task-agnostic prefix tuning for program and language generation," in *Proceedings of the Findings of the 61st Association for Computational Linguistics*. Toronto, Canada: Association for Computational Linguistics, July 9-14 2023, pp. 5282-5297.
- [43] Q. Chen, X. Xia, H. Hu, D. Lo, and S. Li, "Why my code summarization model does not work: Code comment improvement with category prediction," *ACM Transactions on Software Engineering and Methodology*, vol. 30, no. 2, pp. 25:1-25:29, 2021.
- [44] F. Mu, X. Chen, L. Shi, S. Wang, and Q. Wang, "Developer-intent driven code comment generation," in *Proceedings of the 45th International Conference on Software Engineering*. Melbourne, Australia: IEEE, May 14-20 2023, pp. 768-780.
- [45] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin *et al.*, "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.
- [46] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.
- [47] L. Tunstall, N. Lambert, N. Rajani, E. Beeching, T. Le Scao, L. von Werra, S. Han, P. Schmid, and A. Rush, "Creating a coding assistant with starcoder," *Hugging Face Blog*, 2023, <https://huggingface.co/blog/starchat>.
- [48] Bigcode, "Starcodeplus," *Hugging Face Blog*, 2023, <https://huggingface.co/bigcode/starcodeplus>.
- [49] OpenAI, "OpenAI API," site: <https://platform.openai.com/docs/models>, 2015, accessed: 2024-08-07.
- [50] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Proceedings of the 36th Annual Conference on Neural Information Processing Systems*. New Orleans, LA, USA: Curran Associates Inc., November 28 - December 9 2022, pp. 24 824-24 837.
- [51] Y. Wang, Z. Zhang, and R. Wang, "Element-aware summarization with large language models: Expert-aligned evaluation and chain-of-thought method," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*. Toronto, Canada: Association for Computational Linguistics, July 9-14 2023, pp. 8640-8665.
- [52] G. Kim, P. Baldi, and S. McAleer, "Language models can solve computer tasks," in *Proceedings of the 37th Annual Conference on Neural Information Processing Systems*, vol. 36. New Orleans, LA, USA: Curran Associates, Inc., December 10 - 16 2023, pp. 39 648-39 677.
- [53] B. Xu, A. Yang, J. Lin, Q. Wang, C. Zhou, Y. Zhang, and Z. Mao, "Expertprompting: Instructing large language models to be distinguished experts," *CoRR*, vol. abs/2305.14688, no. 1, pp. 1-6, 2023.
- [54] CodeLlama, "Application of codellama," site: <https://huggingface.co/spaces/codellama/codellama-13b-chat/blob/main/app.py>, 2023, accessed: 2024-08-07.
- [55] H. Husain, H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "Codesearchnet challenge: Evaluating the state of semantic code search," *CoRR*, vol. abs/1909.09436, 2019.
- [56] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "UniXcoder: Unified cross-modal pre-training for code representation," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*. Dublin, Ireland: Association for Computational Linguistics, May 22-27 2022, pp. 7212-7225.
- [57] D. Wang, B. Chen, S. Li, W. Luo, S. Peng, W. Dong, and X. Liao, "One adapter for all programming languages? adapter tuning for code search and summarization," in *Proceedings of the 45th International Conference on Software Engineering*. Melbourne, Australia: IEEE, May 14-20 2023, pp. 5-16.
- [58] S. Liu, Y. Chen, X. Xie, J. K. Siow, and Y. Liu, "Retrieval-augmented generation for code summarization via hybrid GNN," in *Proceedings of the 9th International Conference on Learning Representations*. Virtual Event, Austria: OpenReview.net, May 3-7 2021, pp. 1-13.
- [59] M. Allamanis, "The adverse effects of code duplication in machine learning models of code," in *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, 2019, pp. 143-153.
- [60] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating text generation with BERT," in *Proceedings of the 8th International Conference on Learning Representations*. Addis Ababa, Ethiopia: OpenReview.net, April 26-30 2020, pp. 1-14.
- [61] D. Cer, Y. Yang, S.-y. Kong, N. Hua, N. Limtiaco, R. S. John, N. Constant, M. Guajardo-Cespedes, S. Yuan, C. Tar *et al.*, "Universal sentence encoder," *arXiv preprint arXiv:1803.11175*, 2018.
- [62] D. Roy, S. Fakhoury, and V. Arnaoudova, "Reassessing automatic evaluation metrics for code summarization tasks," in *Proceedings of the 29th Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. Athens, Greece: ACM, August 23-28 2021, pp. 1105-1116.
- [63] Y. Zhang, Y. Liu, X. Fan, and Y. Lu, "RetCom: Information retrieval-enhanced automatic source-code summarization," in *Proceedings of the 22nd International Conference on Software Quality, Reliability and Security*. Guangzhou, China: IEEE, December 5-9 2022, pp. 948-957.
- [64] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 23th Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Minneapolis, MN, USA: Association for Computational Linguistics, June 2-7 2019, pp. 4171-4186.
- [65] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese bert-networks," in *Proceedings of the 9th International Joint Conference on Natural Language Processing*. Hong Kong, China: Association for Computational Linguistics, November 3-7 2019, pp. 3980-3990.
- [66] W. J. Conover, *Practical nonparametric statistics*. John Wiley & sons, 1999, vol. 350.
- [67] C. P. Dancy and J. Reidy, *Statistics without maths for psychology*. Pearson education, 2007.
- [68] OpenAI, "Create chat completion," site: <https://platform.openai.com/docs/api-reference/chat/create>, 2024, accessed: 2024-08-07.
- [69] L. Tunstall, N. Lambert, N. Rajani, E. Beeching, T. Le Scao, L. von Werra, S. Han, P. Schmid, and A. Rush, "Starchat-beta," site: <https://huggingface.co/HuggingFaceH4/starchat-beta>, 2023, accessed: 2024-08-07.
- [70] pcuenq, "Usage of codellama," site: <https://huggingface.co/spaces/codellama/codellama-13b-chat/blob/main/app.py>, 2023, accessed: 2024-08-07.
- [71] OpenAI, "Get up and running with the openai api," site: <https://platform.openai.com/docs/quickstart?context=python>, 2024, accessed: 2024-08-07.